

Material de Aula

Curso: Bacharelado em Ciência da Computação

Disciplina: Tecnologia de Orientação à Objetos (TOO)

Professora: Vanessa Lago Machado

Programação Orientada à Objetos (POO)

9. Pilar Abstração: Material de Aula – Classe Abstrata e Método Abstrato**

Compreender o **pilar do Abstração**, explorando o conceito de **Classe Abstrata** e **Método Abstrato**, e aplicar esses conceitos na modelagem de tarefas em Python.

9.1. O que é uma Classe Abstrata?

Uma **classe abstrata** é uma **classe que serve de modelo para outras classes**, mas **não pode ser instanciada diretamente**.

Ela define uma **estrutura comum** (atributos e métodos) que será **compartilhada** pelas classes filhas, mas **deixa a implementação de certos comportamentos** (métodos abstratos) para essas subclasses.

Em Python, as classes abstratas são definidas com a ajuda do módulo abc:

```
from abc import ABC, abstractmethod
```

- ABC → indica que a classe é **abstrata** (não pode ser instanciada).
- @abstractmethod → define **métodos que devem ser obrigatoriamente implementados** pelas subclasses.

9.2. Criando uma Classe Abstrata

Exemplo utilizado em aula — Classe **Tarefa** como modelo base para outros tipos de tarefas:

```
from datetime import datetime
from abc import ABC, abstractmethod

class Tarefa(ABC):
    def __init__(self, nome_tarefa, \
        descricao=None, data_realizacao=None):
        self.nome = nome_tarefa
        self.__concluido = False
        self.__descricao = descricao
        self.__data_realizacao = None
        if data_realizacao:
            self.data_realizacao = data_realizacao

    # Encapsulamento
    @property
    def nome(self):
        return self.__nome

    @nome.setter
    def nome(self, nome_tarefa):
        self.__nome = nome_tarefa.title()

    @property
    def descricao(self):
        return self.__descricao

    @descricao.setter
    def descricao(self, desc):
        self.__descricao = desc

    @property
    def data_realizacao(self):
        return self.__data_realizacao

    @data_realizacao.setter
    def data_realizacao(self, data):
        try:
```

```

        self.__data_realizacao =
            datetime.strptime(data, "%d-%m-%Y")
    except Exception:
        print("Data inválida! dd-mm-aaaa.")

# Método comum
def concluir(self):
    self.__concluido = True

# Método abstrato → deve ser implementado nas subclasses
@abstractmethod
def exibir_dados(self):
    pass

def __str__(self):
    status = "CONCLUÍDO" if \
        self.__concluido else "A FAZER"
    return f"Tarefa: {self.nome} [{status}]"

```

- Tarefa **não pode ser instanciada** diretamente.

Se tentarmos:

```
t = Tarefa("Aula T00")
```

- o Python retornará:

```

TypeError: Can't instantiate abstract class
Tarefa with abstract method exibir_dados

```

- O método `exibir_dados()` é **abstrato**, ou seja, **obrigatório** de ser implementado pelas subclasses.

9.3. Criando Subclasses Concretas

As subclasses **herdam** a estrutura da classe `Tarefa` e **implementam** o método abstrato `exibir_dados()`.

Exemplo – TarefaPessoal

```

from datetime import datetime
from model.Tarefa import Tarefa

class TarefaPessoal(Tarefa):
    def __init__(self, titulo, tipo=None,
        descricao=None, data_realizacao=None):
        super().__init__(titulo, descricao=descricao, \
            data_realizacao=data_realizacao)
        self.tipo = tipo

    @property
    def tipo(self):
        return self.__tipo

    @tipo.setter
    def tipo(self, tp_tarefa):
        self.__tipo = tp_tarefa.strip().title() \
            if tp_tarefa else None

    def exibir_dados(self):
        status = "CONCLUÍDO" if self._Tarefa__concluido else "A FAZER"
        tipo = f"Tipo: {self.tipo}" if self.tipo else ""
        return f"Tarefa Pessoal:\n - {self.nome} [{status}]\n - {tipo}"

```

9.4. Resumo da Aula

As **classes abstratas** são fundamentais para **projetar sistemas orientados a objetos com hierarquia bem definida**, garantindo que todas as subclasses sigam o mesmo padrão de comportamento.

- O uso de ABC e @abstractmethod ajuda a **organizar o código, aumentar a reutilização e reduzir erros de implementação**.